MODULE 03 · DOMAIN 1 - SDLC AUTOMATION

CodeBuild in Depth

The buildspec, what runs when a phase fails, where secrets come from, how builds reach private networks, and how to make them fast.

DOMAIN 1 of 6 EXAM WEIGHT 22% MODULE 3 of 34

admin@sthithapragna.com

A build is a **clean room** that exists for minutes. Everything it needs must be brought in. Everything worth keeping must be carried out.

The life of one build

01
Queued
Waits for capacity.

02
Provisioning
Container starts from the build image.

03
Download source
Git clone or S3 fetch.

04
Your four phases
Install, pre-build, build, post-build.

05
Upload artifacts
Always attempted, even after a failed build.

06
Finalizing
Logs flushed, container destroyed.

ON THE EXAM

Only the middle box is yours. Queue and provisioning time are what Lambda compute and reserved fleets reduce.

What belongs in each phase

PHASE	TYPICAL WORK
`install`	Choose runtime versions. Install tools the image lacks.
`pre_build`	Sign in to ECR. Fetch dependencies. Lint.
`build`	Compile, run unit tests, build the image.
`post_build`	Push the image, package, notify.

> ON THE EXAM
>
> The phases are conventions until something fails. Then the phase a command lives in decides what happens next.

When a phase fails, by default

- A failure in **install** or **pre_build** ends the build. Nothing after it runs.
- A failure in **build** still runs **post_build**. Then artifacts upload is attempted.
- A **finally** block runs even when a command in its phase fails.
- **on-failure** overrides the default: ABORT, CONTINUE, or RETRY with a count and a pattern.

> ON THE EXAM
>
> RETRY variants are recent and run on EC2 compute only. On Lambda compute and reserved capacity, on-failure is ignored.

The post_build trap

```
1  phases:
2    build:
3      commands:
4        - npm test
5        - docker build -t $REPO:latest .
6    post_build:
7      commands:
8        - docker push $REPO:latest
9
10 # the fix: only push from a build that is still succeeding
11       - '[ "$CODEBUILD_BUILD_SUCCEEDING" = "1" ] && docker push $REPO:latest'
```

4, 8 Tests fail, the build phase fails, and post_build still runs the push.

11 CODEBUILD_BUILD_SUCCEEDING is 1 while the build is healthy, 0 once it has failed.

ON THE EXAM

A failed build that still published an image is almost always a push in post_build with no guard.

Getting values into the clean room

SOURCE	BUILDSPEC KEY	SUITABLE FOR SECRETS?
Plain variables	env: variables	No. Visible in the console and API
Parameter Store	env: parameter-store	Yes, as a SecureString
Secrets Manager	env: secrets-manager	Yes, with rotation
Project settings	Type PARAMETER_STORE or SECRETS_MANAGER	Yes, same stores
Built-in	CODEBUILD_* variables	Metadata such as build ID and commit

ON THE EXAM

A secret in a plain environment variable is exposed to anyone who can view the project. That option is always wrong.

Who the build is, and what it may touch

- Every build runs as the project's **service role**, not as the person who started it.
- Reading a secret needs the fetch permission **and** kms:Decrypt if a customer managed key encrypts it.
- Pushing to ECR needs ecr:GetAuthorizationToken plus the layer and image upload actions.
- Reaching another account means assuming a role there from inside the build.

> ON THE EXAM
>
> Access denied inside a build: fix the service role, or a key policy. Never the developer's own permissions.

Default network, or your VPC?

DEFAULT

AWS-managed network

- Internet access works out of the box
- Cannot reach private resources in your VPC
- Right for most builds
- Nothing to configure

VPC CONFIGURATION

ENIs in your subnets

- Reaches private RDS, caches, internal APIs
- Loses internet unless routed through NAT
- Needs private subnets and security groups
- Public subnets do not give it internet

THE TIEBREAKER

Build in a VPC cannot download packages: the subnets need a NAT gateway route, or VPC endpoints for AWS services.

Choosing compute

COMPUTE	STRENGTH	LIMITS
EC2, on demand	Everything works: Docker, root, caching	Queue and provisioning time
Lambda	Starts in seconds, billed per second	15 min, no Docker, no root, no cache
Reserved capacity fleet	Warm hosts, no provisioning wait	You pay while it is idle
Arm (Graviton)	Lower cost for compatible builds	Tools must support Arm
GPU	Machine learning and graphics builds	Higher cost

ON THE EXAM

Needs Docker, root or caching: EC2. Short, simple, frequent builds: Lambda. Consistent speed at scale: a reserved fleet.

Caching: four kinds

CACHE	WHERE IT LIVES	BEST FOR
S3	A bucket, shared by every host	Dependencies, infrequent builds
Local: source	On the build host	Large Git repositories
Local: Docker layer	On the build host	Image builds. Needs privileged mode
Local: custom	On the build host	Directories you name in the buildspec

ON THE EXAM

Local cache is best effort and host-bound: great for frequent builds, useless for rare ones. S3 cache always persists.

Building container images

- **Privileged mode** must be on for the Docker daemon to build images.
- Sign in to ECR in pre_build with a short-lived token from the service role.
- Tag with the commit or build ID, not only **latest**. Deployments should name an exact image.
- Lambda compute cannot build images at all. Windows cannot use privileged mode.

> ON THE EXAM
>
> 'Cannot connect to the Docker daemon' in the build log: privileged mode is off.

Carrying things out: artifacts

- One **primary** artifact, plus named **secondary artifacts** for separate outputs.
- **base-directory** sets the root; **discard-paths** flattens folder structure.
- Artifacts are encrypted with the project's KMS key. A customer managed key is needed for cross-account use.
- Inside CodePipeline, output goes to the pipeline's artifact store, not the project's own setting.

> ON THE EXAM
>
> One build, several separately deployable outputs: secondary artifacts. Not several projects, and not a Lambda to split a zip.

Carrying things out: test reports

- A **reports** section sends results to a **report group**, viewable per test with trends.
- Test formats include JUnit XML, Cucumber, NUnit, TestNG and Visual Studio TRX.
- Coverage formats include JaCoCo, Cobertura, Clover, SimpleCov and LCOV.
- Reports are still published when tests fail. That is when you need them.

> ON THE EXAM
>
> 'See which tests failed without reading logs' or 'track coverage over time': a report group, not a custom parser.

Starting builds without a pipeline: webhooks

- A project can have its own **webhook** on GitHub, GitLab or Bitbucket, and report status back.
- **Filter groups** decide which events build. Groups are OR'd; filters inside a group are AND'd.
- Every group needs an **EVENT** filter: PUSH, PULL_REQUEST_CREATED, PULL_REQUEST_MERGED, and more.
- Others: HEAD_REF, BASE_REF, FILE_PATH, COMMIT_MESSAGE, and **ACTOR_ACCOUNT_ID**.

> ON THE EXAM
>
> Pull request validation that reports a pass or fail to the pull request: a CodeBuild webhook. No pipeline needed.

Two filter groups

```
1 [
2   [ { "type": "EVENT",    "pattern": "PULL_REQUEST_CREATED, PULL_REQUEST_UPDATED" },
3     { "type": "BASE_REF", "pattern": "^refs/heads/main$" },
4     { "type": "ACTOR_ACCOUNT_ID", "pattern": "^(1001|1002|1003)$" } ],
5
6   [ { "type": "EVENT",    "pattern": "PUSH" },
7     { "type": "HEAD_REF", "pattern": "^refs/heads/main$" } ]
8 ]
```

2-4 Group 1: a pull request into main, AND opened by one of three maintainers.

6-7 Group 2: any push to main. Either group starts a build.

ON THE EXAM

On a public repository, anyone can open a pull request, and the build runs their code with your service role.

Webhook on CodeBuild, or trigger on CodePipeline?

CODEBUILD WEBHOOK

One build, status back to Git

- Validate a pull request before merge
- Pass or fail shown on the pull request
- No stages, no deployment
- Fastest feedback for developers

CODEPIPELINE TRIGGER

A full release

- Build, test, approve, deploy
- Artifacts promoted across stages
- Execution modes and stage conditions
- What runs after the merge

THE TIEBREAKER

Before merge: CodeBuild webhook, or a PARALLEL pull request pipeline. After merge: the release pipeline.

Batch builds: one definition, many builds

TYPE	WHAT IT DOES	EXAMPLE
Build list	Runs listed builds in parallel	Build for three platforms at once
Build matrix	Every combination of variables	Three runtimes by two operating systems
Build graph	Builds with dependencies between them	Libraries first, then services
Build fanout	Splits one test suite across builds	Shard 2,000 tests over ten builds

ON THE EXAM

'Test on several runtime versions from one project' is a build matrix. 'Tests take too long to run' can be fanout.

Runners: keep the workflow, change the compute

- CodeBuild can host **GitHub Actions**, GitLab and Buildkite runners.
- The workflow file stays in the repository. The job runs on CodeBuild, under its service role.
- GitHub jobs arrive through the **WORKFLOW_JOB_QUEUED** webhook event.
- No long-lived AWS access keys stored in the Git provider's secrets.

> ON THE EXAM
>
> 'Keep existing GitHub Actions workflows but run them in AWS with IAM, not stored keys': CodeBuild-hosted runners.

Watching and debugging builds

- Logs go to **CloudWatch Logs**, to **S3**, or both. Secret values are masked.
- Build and phase **state changes** go to EventBridge, like pipeline events.
- **Session Manager** can pause a running build so you can shell in and look around.
- Build timeout runs from 5 minutes to 36 hours. Queued builds have a separate timeout.

> ON THE EXAM
>
> Notify on failed builds: EventBridge. Interactive debugging of a failing build: Session Manager, not SSH keys.

A buildspec runs unit tests and docker build in the build phase, then docker push with the latest tag in post_build. An ECS service deploys whatever image is tagged latest.

Yesterday a build failed its unit tests, but a new image was still pushed and deployed, causing an outage. The team wants the smallest change that stops failed builds from publishing images.

What should the team change?

A Move the docker push command into the install phase

B Set on-failure: CONTINUE on the build phase

C Add a finally block to the build phase that runs docker push

D In post_build, push only when CODEBUILD_BUILD_SUCCEEDING equals 1

◆ PAUSE AND DECIDE

What should the team change?

A Push in install

Install runs before the image even exists. There would be nothing to push.

B on-failure: CONTINUE

Makes it worse: every later phase runs regardless. That is already why the push happened.

C finally block pushes

A finally block runs even when a command fails. It guarantees the broken push.

D Guard on CODEBUILD_BUILD_SUCCEEDING

The variable is 0 once the build has failed, so the push is skipped. One line.

THE SIGNAL

Build failed, image still published: post_build runs after a failed build phase. Guard it.

> An open-source project has a public GitHub repository. A CodeBuild webhook builds every pull request. The project's service role can read a Secrets Manager secret used to publish documentation.
>
> The security team is concerned that anyone can open a pull request containing code that prints the secret. Pull requests from the five maintainers must still be built automatically.

What should the DevOps engineer do?

A. Add an ACTOR_ACCOUNT_ID filter matching the maintainers' account IDs to the pull request filter group

B. Move the secret into a plaintext environment variable so CodeBuild masks it in logs

C. Run the build in private subnets with no route to the internet

D. Turn off CloudWatch Logs for the project so printed values are not stored

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A ACTOR_ACCOUNT_ID filter

Untrusted pull requests never start a build, so their code never runs with the role. Maintainers still build.

B Plaintext variable

Backwards. Plain variables are visible to anyone who can view the project, and are not masked.

C No internet route

The attacker's code still runs and can still read the secret. It also breaks dependency downloads.

D Turn off logs

The code could send the secret elsewhere. Hiding the evidence is not a control.

THE SIGNAL

Untrusted code with a trusted role: stop the build from starting, rather than limiting what it can do once running.

A team adds a VPC configuration to a CodeBuild project so integration tests can reach a private Amazon RDS database. The project uses two public subnets that route to an internet gateway.

Now the install phase times out while downloading npm packages. The database connection itself works.

How should the team fix the build?

A Enable privileged mode on the project

B Change the compute type to a larger size

C Use private subnets whose route table sends internet traffic through a NAT gateway

D Turn on auto-assign public IPv4 addresses for the two public subnets

◆ PAUSE AND DECIDE

How should the team fix the build?

A **Privileged mode**

Enables the Docker daemon. Nothing to do with network routing.

B **Larger compute**

A timeout here is a missing route, not a slow machine.

C **Private subnets via NAT**

Build interfaces never get public addresses, so internet traffic must leave through NAT. The database stays reachable.

D **Auto-assign public IPs**

The tempting fix, but CodeBuild's network interfaces do not receive public addresses. It changes nothing.

THE SIGNAL

VPC build, internet timeout, public subnets: the classic. CodeBuild in a VPC reaches the internet only through NAT.

> To cut queue and start-up time, a team moved all of its CodeBuild projects to Lambda compute. Short test builds are now much faster.
>
> However, every project that builds a container image now fails. The team wants image builds working again while keeping start-up time low.

What should the team do?

A Increase the memory allocated to the Lambda compute for the image projects

B Move only the image projects to EC2 compute on a reserved capacity fleet

C Enable privileged mode on the Lambda compute image projects

D Enable a local Docker layer cache on the Lambda compute image projects

◆ PAUSE AND DECIDE

What should the team do?

A. More Lambda memory

Lambda compute cannot run Docker at any size. Memory is not the constraint.

B. Image projects to a reserved EC2 fleet

EC2 compute supports Docker. Warm reserved hosts remove the provisioning wait. Test builds stay on Lambda.

C. Privileged mode on Lambda

Not supported on Lambda compute. The setting that fixes Docker on EC2 does not exist here.

D. Docker layer cache on Lambda

Lambda compute supports no caching, and a cache cannot help a build that cannot run Docker.

THE SIGNAL

Lambda compute fails at Docker, root tools, caching and anything over 15 minutes. Split the workload, do not abandon Lambda.

Module 3 in five lines

- A failed build phase still runs post_build. Guard pushes with CODEBUILD_BUILD_SUCCEEDING.
- Secrets come from Parameter Store or Secrets Manager, fetched by the service role, never plaintext.
- VPC builds reach the internet only through NAT. Public subnets do not help.
- Lambda compute is fast but has no Docker, root, cache, or builds over 15 minutes.
- Filter groups: OR between groups, AND within. ACTOR_ACCOUNT_ID protects public repositories.

> ON THE EXAM
>
> Next, module 4: testing in the pipeline. Which tests run where, how they gate a release, and how to test at scale.